

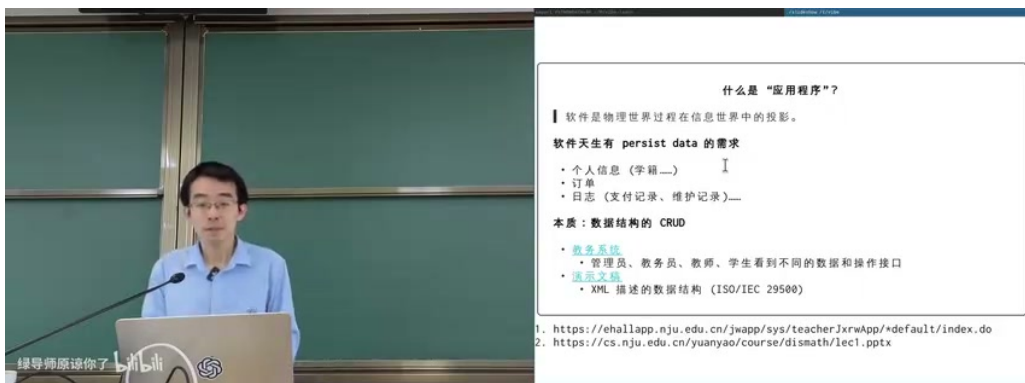
课程笔记

27 - 数据库系统

2026 南京大学操作系统原理

lecture-to-notes

2026 年



视频作者/频道：南京大学操作系统原理（蒋炎岩）

发布日期：课程本地视频

视频时长：01:23:10

视频链接：未填写

目录

1 从文件系统到数据库系统	3
1.1 应用程序是物理过程的信息投影	3
1.2 文档也是数据结构	4
1.3 本章小结	4
2 方案一：用目录和文件实现应用数据	4
2.1 优势：透明、可组合、agent friendly	5
2.2 代价：多文件原子性和性能	5
2.3 本章小结	6
3 方案二：在一个文件上构造数据结构	6
3.1 mmap：把文件映射成内存	6
3.2 Write-Ahead Logging	7
3.3 Single-Level Store 的想象	8
3.4 本章小结	8
4 关系型数据库：二维表与查询语言	8
4.1 从指针到关系	9
4.2 Join：看似笨重，实际可优化	10
4.3 ACID 与事务	10
4.4 本章小结	11
5 数据库作为系统：优化器、并发控制和 SQLite	11
5.1 查询优化：数据库里的编译器	12
5.2 并发控制：一把大锁的语义，高并发的实现	12
5.3 SQLite：无处不在的小数据库	13
5.4 本章小结	13
6 规模化：从 SQL 到 Key-Value、Redis、NoSQL 和 NewSQL	14
6.1 透明 SQL 的边界	14
6.2 Key-Value：减少表达能力换取可扩展性	14
6.3 Redis：把高频变化先放进内存	15
6.4 NoSQL 与 NewSQL	16
6.5 本章小结	17
7 向量数据库：AI 时代的新索引	17
7.1 从精确查询到相似查询	17
7.2 结构化数据不会消失	18
7.3 本章小结	18

8 总结与延伸 18

8.1 讲者结论 18

8.2 概念压缩 19

8.3 实践 takeaways 19

8.4 拓展阅读 19

8.5 最终小结 20

1 从文件系统到数据库系统

上一讲结束在文件系统实现：磁盘上的数据结构如何用目录、inode、块分配表、日志等机制组织起来。这一讲顺势提出一个更面向应用的问题：既然文件系统已经能保存字节流和目录树，应用程序还需要什么样的持久化抽象？

核心观点

应用程序管理的数据，本质上是一个持久化的数据结构；应用对数据的操作，本质上是数据结构的增删改查（CRUD）。数据库系统就是为这种持久化数据结构提供更高层、更可靠、更可扩展的实现。

讲者用南大教务系统、PowerPoint 文档和课程网站作为连续例子：一个网页中的按钮、表单和列表不是凭空存在的，而是后端数据结构某些字段、记录和关系的投影。用户看见的是界面，程序真正读写的是文件系统或数据库中的数据。

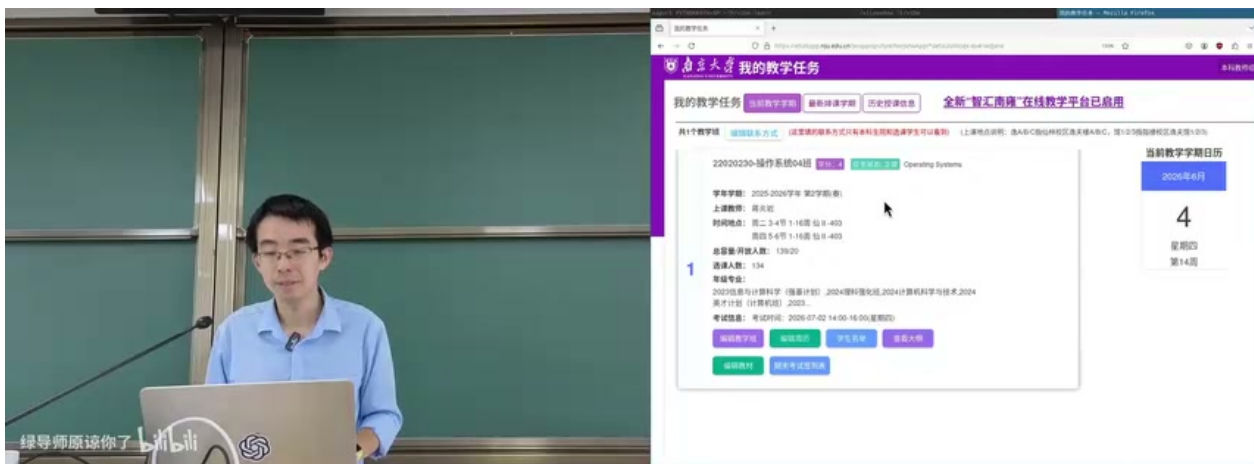


图 1: 教务系统中的课程记录：课程名、授课类型、时间安排和操作按钮都对应后端数据结构中的字段和操作。¹

1.1 应用程序是物理过程的信息投影

软件可以看作物理世界过程在信息世界里的投影：订单、课程、选课、支付、设置、聊天记录、论文阅读进度，都会被压缩成有限的信息并被程序维护。这个投影天然有损，但只要应用希望用户下一次打开时还能看到结果，它就必须让某些状态持久化。

从操作系统视角看，这里的关键不是“界面多漂亮”，而是三个问题：

- 数据用什么抽象表示：文件、目录、表、键值、文档、向量，还是某种专门格式。
- 操作用什么语言表达：系统调用、脚本、SQL、事务、查询 API。

¹视频画面时间区间：00:04:00–00:04:05。

- 失败时如何恢复：进程崩溃、机器断电、磁盘损坏、并发冲突是否会破坏数据结构不变量。

1.2 文档也是数据结构

PowerPoint、Word 这样的文件看似是一个整体，其实常常是一个压缩包，内部包含 XML、图片、主题、布局和元数据。把 .pptx 解开后可以看到类似目录树的数据结构：每一页幻灯片、每张图片、每个 layout 都是可定位、可修改的对象。

文件格式的系统含义

一个“文件”可以只是持久化数据结构的封装边界。对用户来说，.pptx 方便复制；对程序来说，它内部是树、表、引用和资源文件。理解这一点后，数据库系统就不再神秘：它也是在持久化介质上维护数据结构，只是抽象更统一，恢复和并发控制更强。

1.3 本章小结

数据库系统不是文件系统之外的孤岛，而是文件系统之上的一类应用级持久化抽象。它要解决的问题可以压缩为一句话：如何让应用程序把复杂数据结构可靠、高效、并发地保存下来，并在崩溃后仍然保持可恢复的不变量。

2 方案一：用目录和文件实现应用数据

最直接的方案是把应用状态映射到文件系统：学生一个目录、课程一个目录、选课记录一个 Markdown 或 JSON 文件，引用关系用符号链接表示。这种方法非常符合 Unix philosophy：一切都是文件，find、grep、wc、xargs、脚本和 coding agent 都能直接操作。

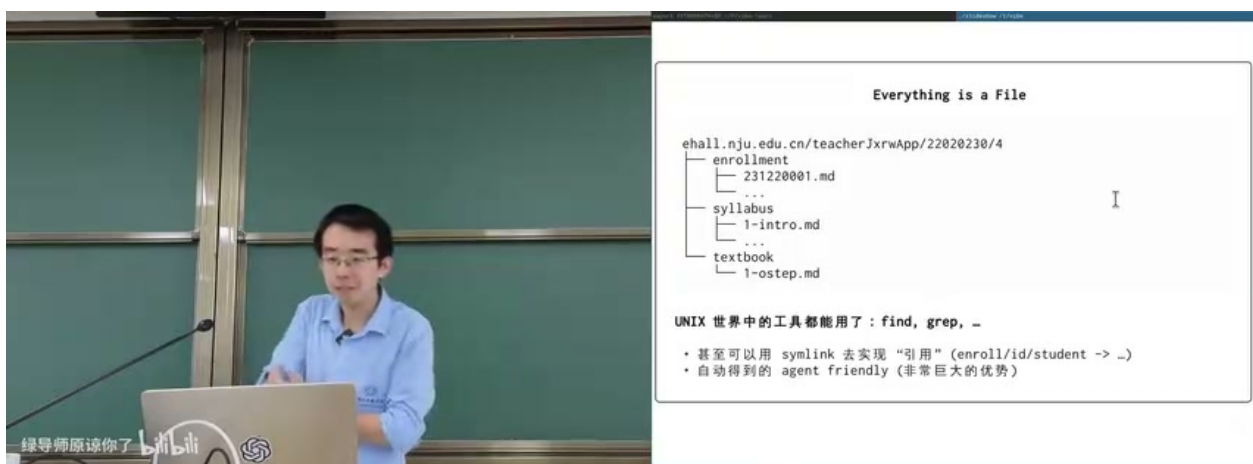


图 2: 把课程网站建模为目录树：课程目录下有选课记录、课程说明、教材等普通文件，Unix 工具可以直接查询和修改。²

²视频画面时间区间：00:09:50–00:09:55。

2.1 优势：透明、可组合、agent friendly

目录和文件作为数据库有三个突出优点。

- **透明**：数据就是普通文件，出问题时可以直接进入目录查看、修补、统计。
- **可组合**：所有 Unix 工具天然可用。统计 accepted 提交数、重判某次作业、删除结果触发重跑，都可以用小脚本完成。
- **对 agent 友好**：文件系统是 coding agent 最熟悉的公共接口。论文库、课程提交、实验结果都可以变成普通目录，让自动化工具抓取、OCR、摘要、索引。

```
1 find M5 -name '*.result' -print0 |  
2   xargs -0 grep '"Accepted"' |  
3   wc -l
```

Listing 1: 文件系统数据库中的典型查询：统计结果文件

这个例子体现了一种很强的工程直觉：如果系统规模不大，且单次业务操作的代价远大于文件系统元数据开销，那么用文件和目录可以快速构建一个可维护、可热修、可观察的信息系统。

2.2 代价：多文件原子性和性能

问题从“一个文件”扩展到“多个文件的一致修改”时开始变难。文件系统通常保证自身元数据不坏，但它不知道应用数据结构的不变量。例如批量导入 100 个学生选课记录时，如果写到第 50 个文件机器崩溃，文件系统可以保持目录结构不损坏，但应用语义已经不一致。



图 3: 目录/文件方案的优缺点：工具友好、agent 友好，但多文件写入的一致性和系统调用成本需要应用自己承担。³

³视频画面时间区间：00:13:40–00:13:45。

文件系统一致性不等于应用一致性

文件系统的 crash consistency 只保证文件系统结构可恢复，不保证应用层“100 个记录要么全写入、要么全不写入”。应用若需要这样的 all-or-nothing 语义，就必须自己设计日志、临时目录加 rename、校验和、重放或 fsck 机制。

性能上，文件系统还会引入路径解析、权限检查、目录项和 inode 元数据维护、修改时间更新、页缓存管理等成本。这些开销在小系统中可以忽略，但在高并发、大量细粒度记录读写时会成为瓶颈。

2.3 本章小结

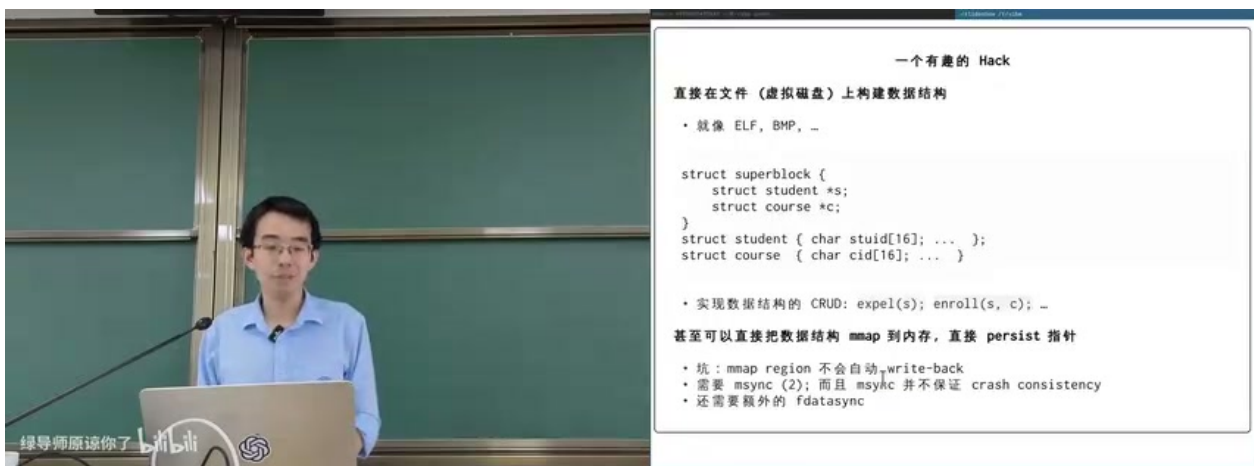
用目录和文件做数据库是完全可行的，尤其适合小规模、可观察性优先、自动化工具参与度高的系统。但它把多对象事务、崩溃恢复和性能优化留给应用程序。随着规模和可靠性要求上升，我们自然会寻找更接近“数据结构本身”的持久化方式。

3 方案二：在一个文件上构造数据结构

如果文件系统的目录、路径和元数据太重，可以退回到更底层的抽象：一个文件就是一个可随机读写的 byte array。既然文件系统能在磁盘 byte array 上构造目录树，应用也能在一个文件 byte array 上构造自己的 superblock、链表、索引和空闲表。

3.1 mmap：把文件映射成内存

讲者演示了一个“山寨教务数据库”：把数据库文件 mmap 到固定虚拟地址，使文件中的指针值在每次运行时都仍然有效。这样，文件里的 struct student *next 可以像普通内存链表一样被 C 代码操作。



一个有趣的 Hack

直接在文件（虚拟磁盘）上构建数据结构

- 就像 ELF, BMP, ...

```
struct superblock {
    struct student *s;
    struct course *c;
};
struct student { char stuid[16]; ... };
struct course { char cid[16]; ... }
```

- 实现数据结构的 CRUD: expel(s); enroll(s, c); ...

甚至可以直接把数据结构 mmap 到内存，直接 persist 指针

- 坑：mmap region 不会自动 write-back
- 需要 msync (2); 而且 msync 并不保证 crash consistency
- 还需要额外的 fdatsync

图 4: 在一个文件中直接实现 superblock、student/course 结构和持久化指针：思路优雅，但 mmap、msync 和 crash consistency 之间仍有坑。⁴

```

1 struct superblock {
2     struct student *students;
3     struct course *courses;
4 };
5
6 void add_student(struct superblock *sb, struct student *s) {
7     s->next = sb->students;
8     sb->students = s;
9 }

```

Listing 2: 在文件映射区域中维护链表的直觉模型

这个想法有吸引力：持久化数据结构的代码和内存数据结构几乎一样。但是它隐藏了几个系统层陷阱。

- 写入 CPU cache 或 DRAM 后，不会立刻稳定落盘。
- `msync` 只是请求把映射页同步回文件，和完整的事务提交不是一回事。
- 为了 crash consistency，仍然需要 `fdatsync`、日志和恢复协议。
- 固定地址映射依赖虚拟地址空间布局；如果换地址，还要修补指针或改用偏移量。

3.2 Write-Ahead Logging

为了让文件内数据结构支持崩溃恢复，可以引入 write-ahead log (WAL)：在真正修改数据结构之前，先把即将执行的操作写入日志并确保落盘；系统重启时，如果发现日志中有未完成操作，就重放它，使数据结构回到一致状态。

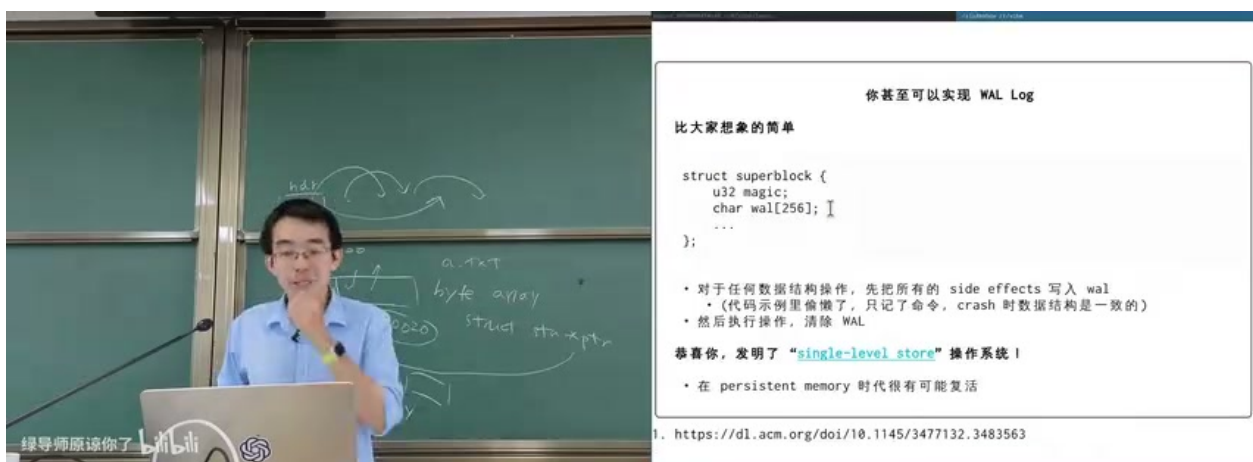


图 5: 一个最小 WAL 设计：superblock 中预留日志槽，先持久化命令，再修改数据结构，重启时重放未完成操作。⁵

⁴视频画面时间区间：00:19:45–00:19:50。

⁵视频画面时间区间：00:29:45–00:29:50。

WAL 的顺序约束

WAL 的关键不在“有一个日志文件”，而在持久化顺序：日志记录必须先于数据结构修改稳定落盘。否则崩溃后系统既不知道要重放什么，也无法判断部分写入是否完成。

抽象地说，一次操作可分为四步：

1. 写入日志：记录命令、参数、校验和或 redo 信息。
2. 强制日志落盘：确保重启后能看见该日志。
3. 修改真实数据结构：更新链表、表项、索引或空闲表。
4. 清除或推进日志：表示该操作已经完成，不需要再重放。

这已经非常接近数据库和文件系统日志的核心思想。差别在于工业数据库会批处理、并发化、维护多个日志段、处理 undo/redo、压缩、检查点、页缓存、索引和事务隔离。

3.3 Single-Level Store 的想象

如果不仅应用数据结构，连整个进程内存都能持久化，就得到 single-level store 的思想：内存与存储不再是两层抽象，而是一个可持久保存的地址空间。持久内存、非易失内存和操作系统恢复机制让这个方向具有研究价值，但也把指针、cache、同步、崩溃一致性、并发恢复等问题推向更复杂的系统设计。

3.4 本章小结

在一个文件中构造数据结构让应用摆脱文件系统目录层的额外开销，但它也把持久化顺序、同步、日志和恢复完全暴露给程序员。由此自然引出数据库系统的野心：让所有应用都能使用通用、高性能、强一致的持久化数据结构，而不是每个应用都重写一套 WAL 和恢复协议。

4 关系型数据库：二维表与查询语言

关系型数据库的革命性不只是“保存数据”，而是把各种应用数据统一到表、行、列和关系上，用声明式查询语言表达派生事实，再由数据库系统负责优化、并发控制和崩溃恢复。

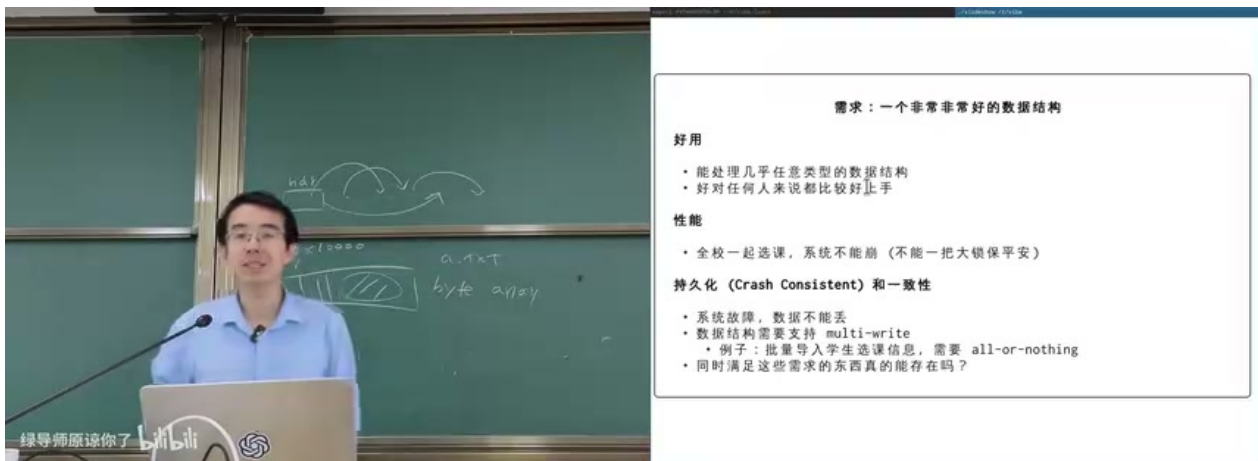


图 6: 关系型数据库的需求: 好用、性能强, 并在多写入和系统崩溃时保持 crash consistency。
6

4.1 从指针到关系

在内存数据结构课里, 我们习惯用指针表达关系: student 结构体里有课程链表, course 结构体里有学生链表。但数据库追求的是通用抽象。学生、课程、选课三种事实可以分别放入三张表, 关系不再由内存指针表达, 而由主键、外键和 join 表达。

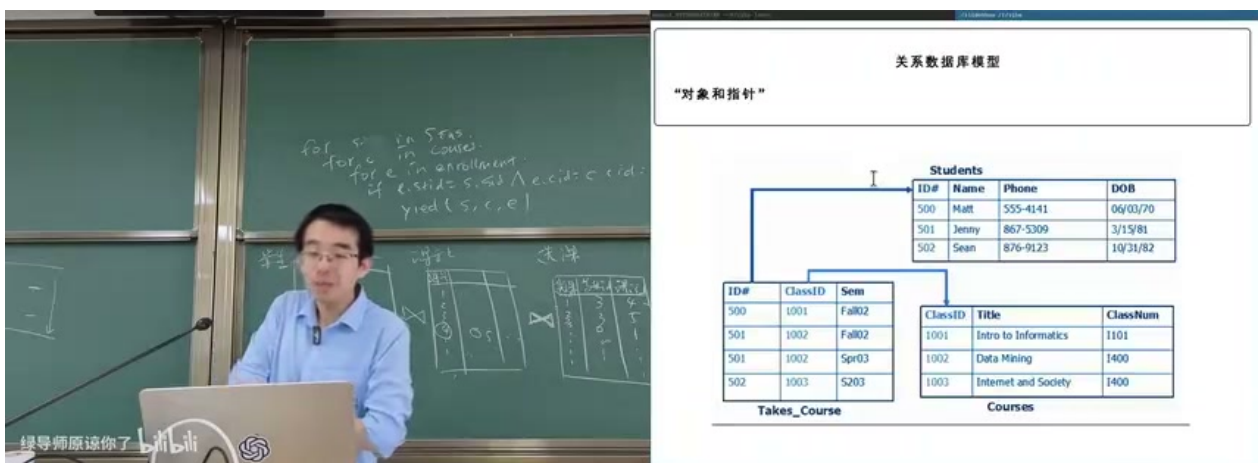


图 7: 学生、课程和选课关系的规范化表结构: 多对多关系不嵌入某个结构体, 而由 Takes_Course 表连接。⁷

关系模型的抽象力量

关系模型把“世界上的事实”分解成最小记录, 再用关系代数从这些事实推出派生事实。它刻意把“数据怎么组织”与“用户想问什么”分离: 用户写查询, 数据库决定如何执行。

⁶ 视频画面时间区间: 00:35:35–00:35:40。

⁷ 视频画面时间区间: 00:46:00–00:46:05。

4.2 Join: 看似笨重, 实际可优化

如果要查询姓张学生选了哪些课, 可以把 `Student`、`Takes_Course`、`Courses` 做笛卡尔积, 再用条件筛选匹配的 ID。这看似是三重循环, 但数据库知道哪些列是 key, 可以把大量循环改写成索引查找。

```
关系数据库查询：指针“配对”

SELECT Courses.Title
FROM Student
JOIN Takes_Course
  ON Student.ID = Takes_Course.ID -- Key
JOIN Courses
  ON Takes_Course.ClassID = Courses.ClassID -- Key
WHERE Student.Name LIKE '张%'
```

ID	Name	Phone
1001	张三	150-4444-4444
1002	李四	860-5555-5555
1003	王五	876-9999-9999

ID	ClassID	Name
1001	1001	Math
1002	1002	Physics
1003	1003	Chem

ClassID	Title	Classroom
1001	Math for Scientists	2001
1002	Physics	1001
1003	Chemical and Society	1002

图 8: SQL join 查询: 用户写的是声明式关系查询, 系统可利用 key 和索引把朴素笛卡尔积改写成高效执行计划。⁸

```
1 SELECT Courses.Title
2 FROM Student
3 JOIN Takes_Course
4   ON Student.ID = Takes_Course.ID
5 JOIN Courses
6   ON Takes_Course.ClassID = Courses.ClassID
7 WHERE Student.Name LIKE '张%';
```

Listing 3: 从学生、选课和课程表查询姓张学生的课程标题

这段 SQL 的教育意义在于: 数据库语言本身是一个小型领域专用编程语言。`SELECT`、`JOIN`、`WHERE`、`INSERT`、`UPDATE`、`DELETE` 都是在表达数据结构上的计算, 剩下的高效执行由数据库完成。

4.3 ACID 与事务

数据库真正让应用程序“省心”的地方, 是它把事务作为并发和崩溃恢复的边界。转账就是经典例子: 从 A 账户扣 100 元、给 B 账户加 100 元, 中间还可能有检查、条件分支和失败。事务要求这些操作要么全部生效, 要么全部不生效。

⁸视频画面时间区间: 00:46:30–00:46:35。

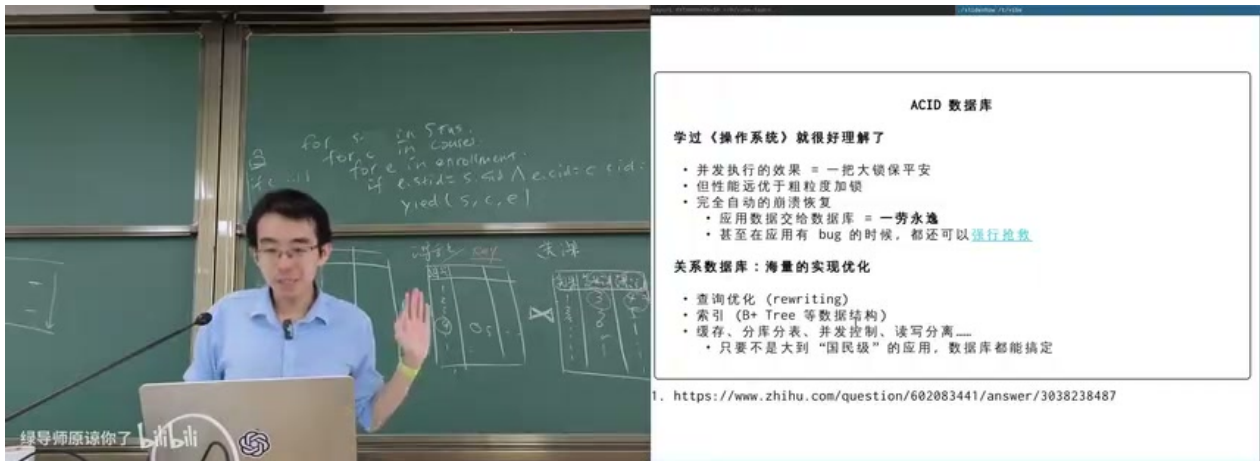


图 9: ACID 数据库的核心承诺：应用看见类似“一把大锁保平安”的事务语义，数据库内部承担高性能并发和崩溃恢复。⁹

ACID 四个字母常见解释如下：

- **Atomicity**：事务内部多个写入 all-or-nothing。
- **Consistency**：事务前后保持应用定义的不变量。
- **Isolation**：并发事务互不干扰，表现得像某种顺序执行。
- **Durability**：事务提交后，即使系统崩溃，结果也不会丢失。

从操作系统课的角度，最应抓住两点：**serializability** 和 **strong crash consistency**。前者让并发事务可解释为某个全局串行顺序；后者让系统在任意时刻崩溃后都能恢复到某个合法事务边界。

4.4 本章小结

关系型数据库把应用数据压缩为事实表，把查询压缩为声明式程序，把可靠性压缩为事务边界。它的抽象看起来比手写指针结构笨重，却通过优化器、索引、缓存、日志和并发控制实现了好用与高性能的组合。

5 数据库作为系统：优化器、并发控制和 SQLite

数据库系统不是一个简单的文件格式，而是一整套编译器、运行时、缓存、索引和恢复系统。SQL 查询进入数据库后，会被解析、重写、优化，最终变成一个访问 B-tree、LSM-tree、缓冲池和日志的执行计划。

⁹视频画面时间区间：00:52:00-00:52:05。

5.1 查询优化：数据库里的编译器

讲者把数据库和编译器类比：编译器把高级语言重写成高效机器代码；数据库把关系查询重写成高效执行计划。常见优化包括谓词下推、join 顺序选择、索引选择、公共子表达式消除、物化策略选择等。

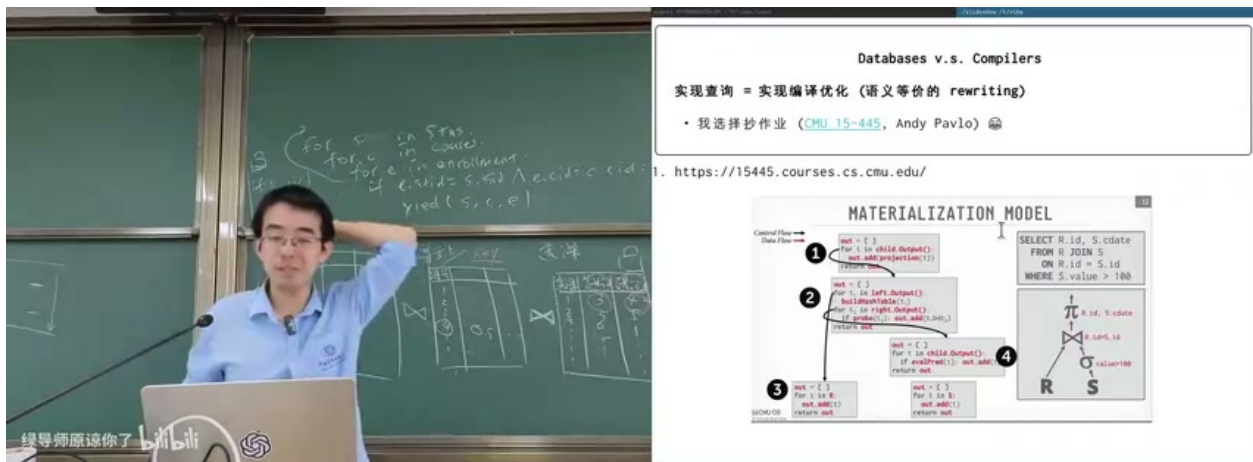


图 10: 数据库和编译器的类比：SQL 查询会经过语义等价的 rewriting，最终形成更高效的执行计划。¹⁰

查询优化的本质

SQL 是“想要什么”的描述，不是“如何循环”的命令。优化器的任务是在保持语义等价的前提下，选择代价更低的执行计划。数据库性能很大一部分来自几十年积累的 rewriting rules、统计信息和物理算子实现。

5.2 并发控制：一把大锁的语义，高并发的实现

应用程序写事务时像在使用一把全局大锁：BEGIN 之后任意读写，最后 COMMIT 或 ABORT。但数据库不能真的只用一把锁，否则高并发性能会崩溃。它需要在内部自动判断哪些读写冲突、哪些可以并行、哪些需要回滚。

两类典型机制是：

- **Two-Phase Locking (2PL)**：读对象上读锁，写对象上写锁；随着事务执行逐步加锁，在提交或回滚时释放。缺点是容易出现 ABBA 死锁，数据库需要动态 deadlock detection 并选择事务回滚。
- **Multi-Version Concurrency Control (MVCC)**：事务开始时看到一个快照，像从当前数据库状态 checkout 一个分支；事务在自己的版本上读写，提交时尝试合并到主线，冲突则回滚某个事务。

¹⁰ 视频画面时间区间：00:55:20–00:55:25。

数据库不能要求业务遵守固定锁顺序

内核代码可以通过 lock ordering 或 lockdep 约束开发者；数据库面对的是用户写的查询和业务逻辑，读写顺序可能由运行时计算决定。因此数据库必须在系统内部处理死锁、回滚和重试，而不能简单禁止某些访问顺序。

5.3 SQLite：无处不在的小数据库

数据库并不只存在于大型服务器中。SQLite 是一个嵌入式 SQL 数据库引擎，库和数据库文件都很小，几乎所有手机、浏览器和应用都可能使用它保存设置、偏好、缓存和小型结构化数据。

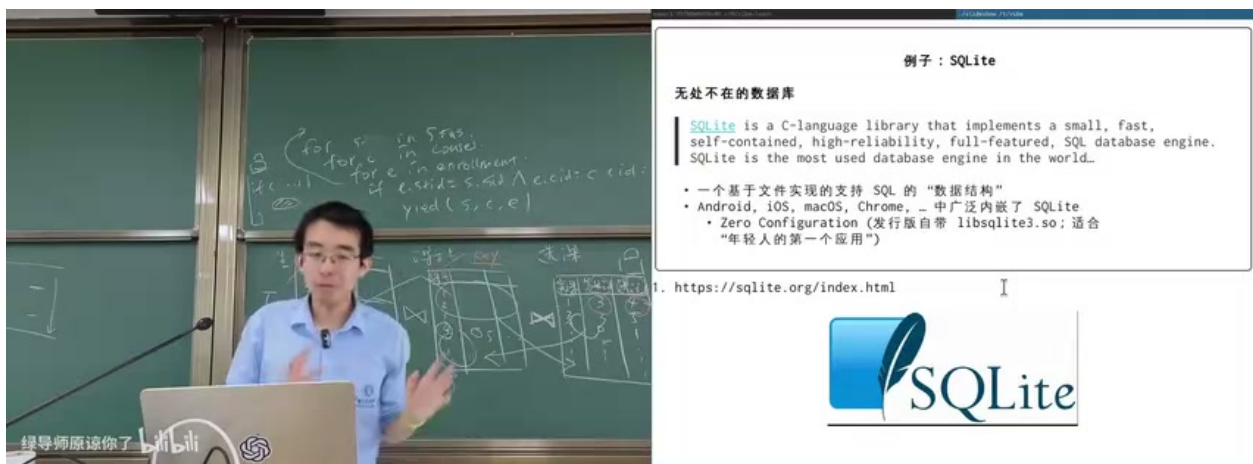


图 11: SQLite：小型、快速、自包含、高可靠的嵌入式 SQL 数据库，是手机和应用设置持久化的常见基础设施。¹¹

这说明数据库系统的价值不只在“支撑百万并发”，还在于为普通应用提供正确的持久化默认值。一个设置开关从 0 变成 1，看似只是写一个配置文件，实际上往往需要原子更新、同步、恢复和查询支持；SQLite 把这些机制打包成一个易用库。

5.4 本章小结

数据库系统把多个系统课主题合在一起：编译器式查询优化、操作系统式缓存和同步、并发程序里的锁与死锁、文件系统式日志和恢复。它给应用的表面接口很简单，内部却是高度工程化的系统集合。

¹¹ 视频画面时间区间：01:02:30–01:02:35。

6 规模化：从 SQL 到 Key-Value、Redis、NoSQL 和 NewSQL

关系数据库足以支撑许多大型应用，但当数据规模、访问频率和全球并发继续增长时，透明 SQL 也会带来新的问题：查询太自由，应用容易写出复杂且难优化的性能炸弹；单机索引、缓存、读写带宽和存储容量也终会触顶。

6.1 透明 SQL 的边界

以微信聊天记录为例，逻辑上它可以是一个巨大表：sender_id、receiver_id、timestamp、content。读取某个会话最近 10 条消息看起来是一条简单 SQL，但在国民级应用中，表大小、索引、分片、缓存、读写分离和热点数据都会使系统复杂度急剧上升。

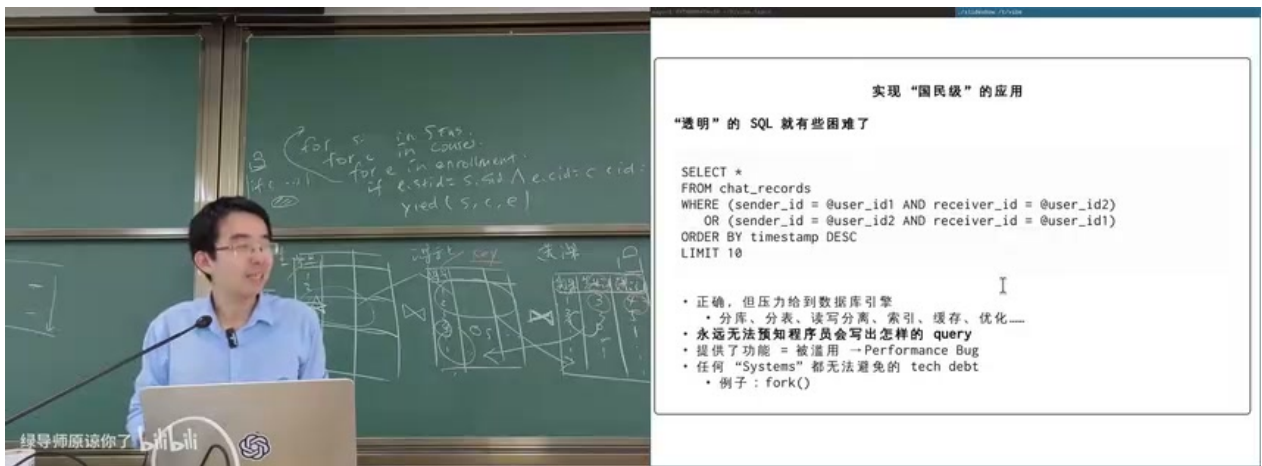


图 12: 聊天记录可以抽象为一张 SQL 表，但超大规模下需要分库分表、索引、缓存和架构约束，否则透明查询会演变成性能债务。¹²

查询自由度也是风险

SQL 让程序员无需关心执行计划，这是巨大生产力；但它也允许写出 50 张表 join、复杂过滤和排序的查询。小规模时这只是慢一点，大规模时会变成事故。系统设计常常需要在表达能力和可预测性能之间取舍。

6.2 Key-Value：减少表达能力换取可扩展性

一种自然做法是提供 SQL 的可扩展子集：key-value store。键经过哈希后可均匀分布到多台机器；值可以是字符串、JSON、列表或 append-only 日志。应用用 key 组织对象，例如 user:{uid}、user:{uid}:like_history、course:{cid}。

¹² 视频画面时间区间：01:05:05–01:05:10。

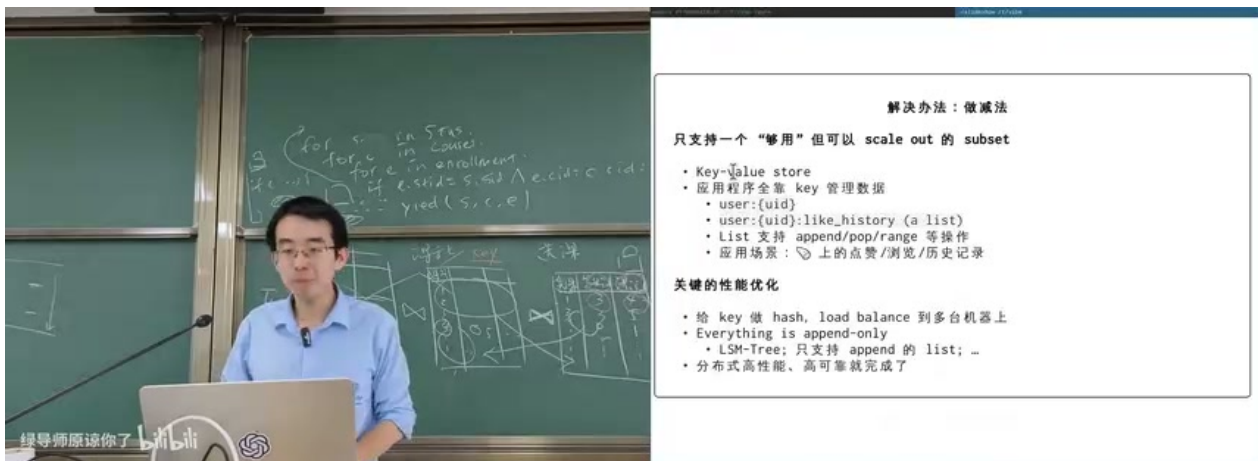


图 13: Key-value 方案减少接口表达能力，只支持够用的数据结构，从而更容易 hash 分片、负载均衡、append-only 存储和分布式扩展。¹³

Append-only 的工程价值

追加比原地修改更容易并发、更容易恢复，也更适合分布式合并。点赞、日志、选课历史、消息流等业务常可先记录事件，再通过 replay 或缓存得到当前视图。

Key-value 并不是万能简化。它把一部分查询能力从数据库转移回应用：程序员需要设计 key schema、反范式数据、缓存一致性和后台聚合。但换来的好处是性能更可预测，分布式分片更直接。

6.3 Redis：把高频变化先放进内存

如果某些数据丢一点不会灾难性失败，例如热点点赞数、短期计数、缓存结果，可以先放入内存 key-value 系统，由后台定期聚合写回持久数据库。Redis 提供 string、list、set、hash、sorted set、stream 等结构，常作为缓存、缓冲层和轻量协调层。

¹³视频画面时间区间：01:09:40–01:09:45。

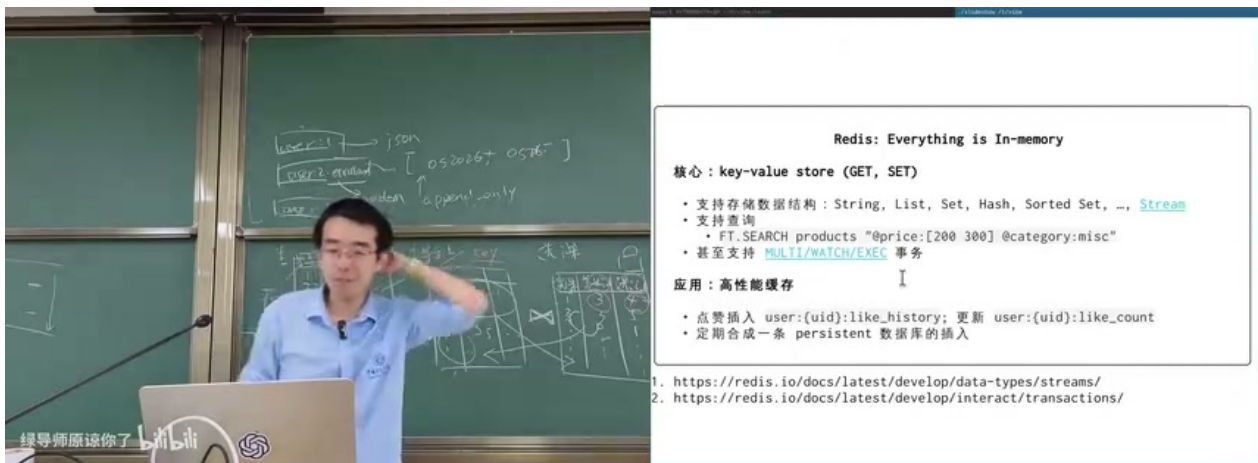


图 14: Redis 作为 in-memory key-value store: 支撑多种数据结构, 可把高频请求缓冲在内存中, 再批量写入持久数据库。¹⁴

一个典型架构是: 请求先更新 Redis 中的计数器或列表, 后台每隔一段时间把增量写入 SQL 数据库。这样底层数据库承受的是聚合后的低频写入, 而不是每一次用户点击产生一次事务。

6.4 NoSQL 与 NewSQL

Key-value 过于简单时, 可以向上扩展为 document database、column database 或兼容 SQL 的分布式数据库。MongoDB 以文档为中心, 值类似 JSON/BSON; Cassandra 采用宽列模型, 强调分布式写入和可用性; TiDB、Google Spanner、CockroachDB 等系统则试图在分布式环境中保留尽可能完整的 SQL 支持。

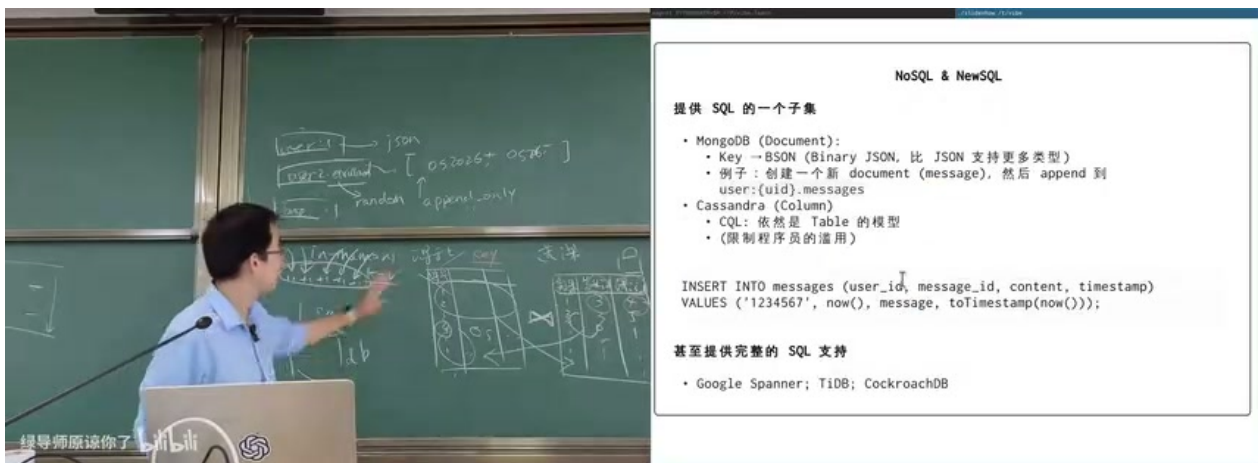


图 15: NoSQL 与 NewSQL: MongoDB、Cassandra 提供 SQL 的子集或替代模型, Spanner、TiDB、CockroachDB 等尝试在分布式系统中保留更完整 SQL。¹⁵

¹⁴视频画面时间区间: 01:15:05–01:15:10。

¹⁵视频画面时间区间: 01:17:10–01:17:15。

没有唯一的“版本答案”

关系数据库、key-value、document、column、Redis、NewSQL 都是在表达能力、性能可预测性、崩溃一致性、分布式扩展、开发成本之间取点。系统设计不是背数据库名词，而是根据业务不变量和负载模型选择合适抽象。

6.5 本章小结

规模化数据库设计的核心不是“SQL 过时了”，而是“抽象要和负载匹配”。SQL 给最大表达能力；key-value 给可预测分片和高吞吐；Redis 给内存级缓冲；NoSQL/NewSQL 在可用性、分布式一致性和查询能力之间寻找工程平衡。

7 向量数据库：AI 时代的新索引

本讲最后引入向量数据库。前面的数据库大多保存结构化事实：表、键、文档、列族。向量数据库保存的是由 embedding model 生成的高维向量，用距离或相似度衡量对象之间的语义接近程度。

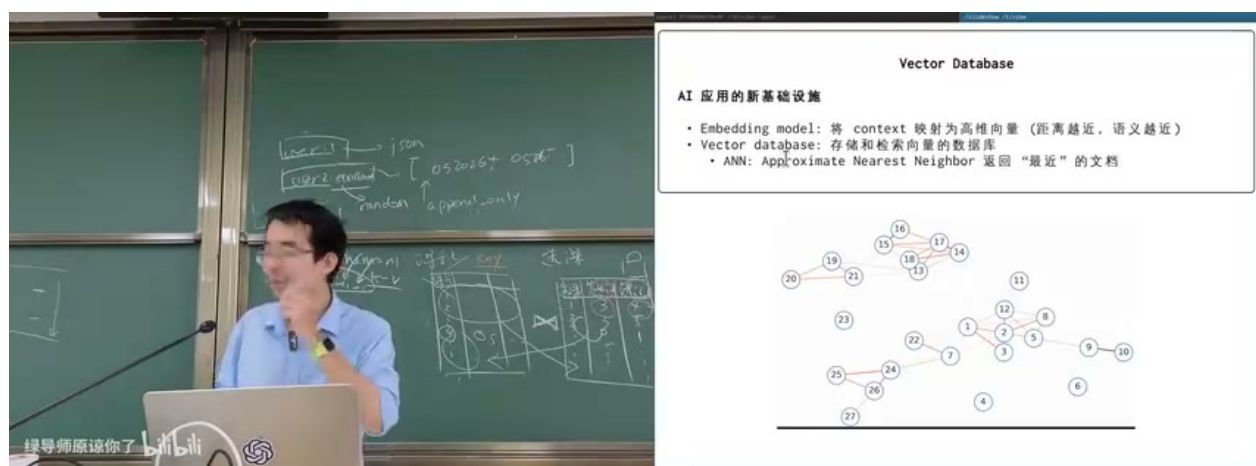


图 16: 向量数据库：embedding model 将上下文映射成高维向量，vector database 支持近邻搜索，返回语义上“最近”的文档。¹⁶

7.1 从精确查询到相似查询

传统 SQL 查询擅长精确条件：某个用户、某个时间范围、某个课程号、某个状态字段。AI 应用常需要的是“语义上相关”的内容：和这篇论文相似的论文、和这个问题相关的讲义、和这个错误日志类似的历史事故。向量把对象投影到高维空间后，相似性可用内积或余弦距离近似表达。

常见度量包括：

¹⁶ 视频画面时间区间：01:18:40–01:18:45。

$$\text{cosine}(x, y) = \frac{x \cdot y}{\|x\| \|y\|}$$

其中：

- x, y 表示两个文档或片段的 embedding 向量；
- $x \cdot y$ 是向量内积；
- $\|x\|, \|y\|$ 是向量长度；
- 值越接近 1，方向越相似，语义上通常越接近。

当文档数量达到百万、千万甚至上亿时，精确计算每一对距离代价太高，因此向量数据库常用 Approximate Nearest Neighbor (ANN) 索引，在速度、内存和召回率之间取舍。

7.2 结构化数据不会消失

向量数据库不是关系数据库的替代品，而是补充。聊天记录、支付流水、课程成绩仍然需要结构化字段、事务和强一致；论文、讲义、图片、日志片段则可以额外关联 embedding，支持语义检索和智能推荐。

未来的信息系统形态

一个现实系统很可能同时拥有关系表、key-value 缓存、对象存储、搜索索引和向量索引。结构化部分保证事实正确，向量部分提供模糊语义连接，两者共同服务智能化应用。

7.3 本章小结

向量数据库把“查询”从精确匹配扩展到语义相似性，但它仍然是在维护某种数据结构的增删改查：插入向量、删除向量、更新元数据、接近邻查询。它没有推翻本讲主线，而是把“应用数据结构”扩展到 AI 时代。

8 总结与延伸

本讲从文件系统自然走向数据库系统。讲者的主线非常清晰：所有应用程序都在管理数据结构，所谓持久化，就是让这些数据结构跨进程、跨重启、跨故障仍然保持意义。

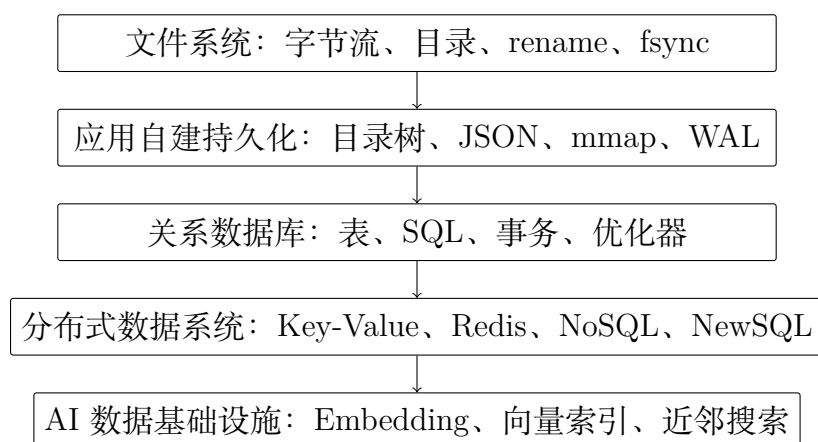
8.1 讲者结论

- 文件和目录可以直接构建小型信息系统，透明、工具友好、agent 友好。
- 在单个文件上用 mmap 和 WAL 可以直接持久化自定义数据结构，但 crash consistency 的顺序约束很难写对。

- 关系型数据库用二维表、关系代数和 SQL 提供通用抽象，并通过 ACID 事务给应用强一致语义。
- 数据库内部是复杂系统：查询优化、索引、缓存、并发控制、日志和恢复共同支撑高性能。
- 大规模系统会根据负载选择 key-value、Redis、NoSQL、NewSQL 或向量数据库等不同抽象。

8.2 概念压缩

可以把本讲压缩成一张从低到高的抽象阶梯：



每一层都没有消灭下一层或上一层。文件系统仍是数据库的落脚点；关系数据库仍是结构化事实的主力；key-value 和 Redis 是性能与规模的工程选择；向量数据库则让“相似性”成为可查询对象。

8.3 实践 takeaways

1. 小系统优先考虑文件和目录：可观察、可修复、可自动化。
2. 一旦需要多对象原子更新，就认真考虑事务数据库，不要低估 crash consistency。
3. SQL 查询写起来简单，不代表执行代价简单；索引、join 顺序和数据规模会改变一切。
4. 缓存和 key-value 不是银弹：它们减少数据库压力，也引入一致性和失效策略问题。
5. AI 应用不是只需要向量库：结构化事实、权限、事务、审计仍然需要传统数据库系统支撑。

8.4 拓展阅读

- E. F. Codd, “A Relational Model of Data for Large Shared Data Banks”。
- CMU 15-445/645 Database Systems, 可用于深入学习查询优化、并发控制和存储引擎。

- SQLite 官方文档，适合理解嵌入式数据库和单文件数据库设计。
- Redis 官方文档，适合理解内存 key-value 数据结构、缓存和 stream。
- 向量数据库与 ANN 索引资料，适合理解 HNSW、IVF、PQ 等近邻检索技术。

8.5 最终小结

数据库系统是操作系统课程中“持久化”主题的自然延伸。它站在文件系统之上，却把文件系统没有直接提供的应用级语义补齐：事务、查询、并发、恢复和扩展。理解数据库，不是背 SQL 语法或产品名，而是理解应用数据结构如何在真实系统里活下来、跑得快、并在失败后恢复到正确状态。